Bi-weekly assignment

Module Details

Module Code	**UFCFK1-15-0**
Module Title	**Fundamentals of Data Science**
Module Tutors	**Saurav Gautam**
Year	**2025-2026**
Component/Element Number	**PSA/Bi-weekly assignment/Regular**
Weighting	**10%**

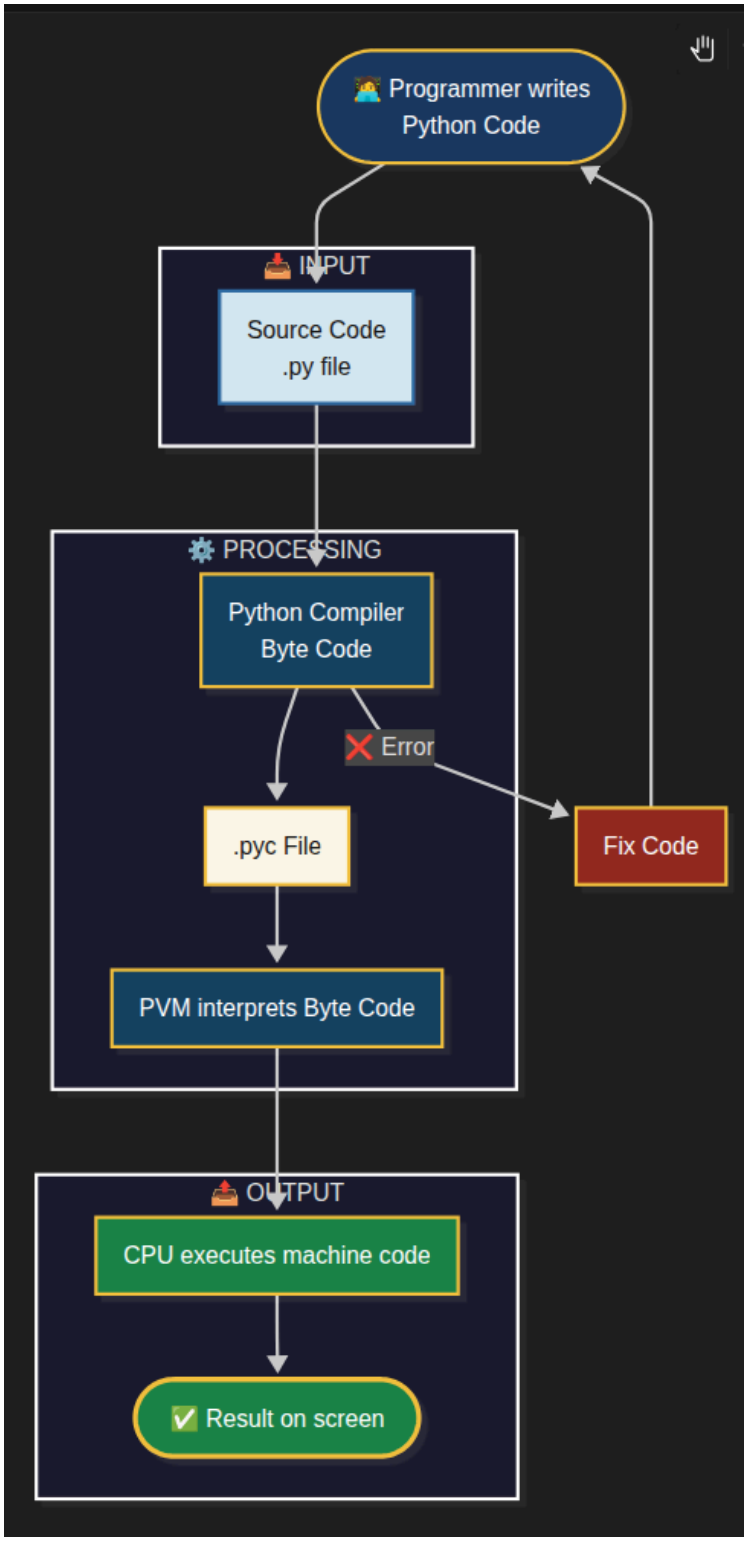
Dates

Submission Date	**17-April-2026**
Submission Place	**Backboard**
Submission Time	**23:59**
Submission Notes	**Submit Gitlab URL**

Submitted By: Adarsha Karki

Student ID: 25068584

1. Create a simple flowchart or diagram to explain the working of Python. The diagram should clearly include input, processing, and output stages. **Marks** [10]



2. Write a program to take **two integer inputs** from the user and display the results of the following operations in a clean format: **Marks** [2*5]

   I. Addition

   II. Multiplication

   III. Division

   IV. Modulus (remainder)

   V. Exponentiation

```
num1 = int(input("Enter the first integer: "))
num2 = int(input("Enter the second integer: "))

add = num1 + num2
mul = num1 * num2
div = num1 / num2 if num2 != 0 else None
mod = num1 % num2 if num2 != 0 else None
exp = num1 ** num2

# Displaying results in a clean format, with special handling for division
by zero cases
print("\n RESULTS ")

print(f"Addition: {num1} + {num2} = {add}")
print(f"Multiplication: {num1} * {num2} = {mul}")
if div is None:
   print("Division: Division by zero is undefined.")
else:
   print(f"Division: {num1} / {num2} = {div:.2f}")
if mod is None:
   print("Modulus: Modulus by zero is undefined.")
else:
   print(f"Modulus: {num1} % {num2} = {mod}")
print(f"Exponentiation: {num1} ** {num2} = {exp}")
```

Output:

```
Enter the first integer: 12
Enter the second integer: 24

 RESULTS
Addition: 12 + 24 = 36
Multiplication: 12 * 24 = 288
Division: 12 / 24 = 0.50
Modulus: 12 % 24 = 12
Exponentiation: 12 ** 24 = 7949684720339084413344153 6
```

3. Write a program that prompts the user to enter a number and determines whether it is **positive, even, positive odd, negative, or negative odd**. **Marks** [2*5]

```
number = int(input("Enter a number: ")) # taking input from the user and converting it to an integer

# Checking if the number is positive, negative, even, or odd and printing the appropriate message
if number > 0 and number % 2 == 0:
   print(f"{number} is a positive even number.")
elif number > 0 and number % 2 != 0:
   print(f"{number} is a positive odd number.")
elif number < 0 and number % 2 == 0:
   print(f"{number} is a negative even number.")
elif number < 0 and number % 2 != 0:
   print(f"{number} is a negative odd number.")
else:
   print(f"{number} is zero, which is neither positive nor negative, and is considered even.")
```

Output:

```
Enter a number: 102
102 is a positive even number.
```

4. Write a program to take a single number as input and display: **Marks** [2*5]

   I. Cube of the number

   II. Cube root of the number

   III. Natural logarithm of the number

   IV. Base-2 logarithm of the number

   V. The number raised to the power of 6

```
e = 2.718281828459045   # Euler's number
ln2 = 0.6931471805599453  # Natural log of 2

number = float(input("Enter a number: "))

# Calculations
cube = number ** 3
cube_root = number ** (1/3)

# Approximation of ln(x)
natural_log = e * (number ** (1/e) - 1)

# log2(x) = ln(x) / ln(2)
base2_log = natural_log / ln2

power_of_6 = number ** 6

print(f"\nNumber entered: {number}")

print("\nResults:")
print(f"Cube of {number}                        : {cube:.4f}")
print(f"Cube root of {number}                   : {cube_root:.4f}")
print(f"Natural logarithm of {number}           : {natural_log:.4f}")
print(f"Base-2 logarithm of {number}            : {base2_log:.4f}")
print(f"{number} raised to the power of 6       : {power_of_6:.4f}")
print()
```

Output:

```
Enter a number: 6

Number entered: 6.0

Results:
Cube of 6.0                      : 216.0000
Cube root of 6.0                 : 1.8171
Natural logarithm of 6.0         : 2.5366
Base-2 logarithm of 6.0          : 3.6595
6.0 raised to the power of 6     : 46656.0000
```

5. Write a program to input the marks of **6 subjects**. Calculate total, average, percentage, and grade according to the rules below: **Marks** [2*5]
    - Distinction: 85% or above
    - First Division: 70% and above
    - Second Division: 55% and above
    - Third Division: 45% and above
    - Fail: Below 45%

```python
# Taking input for marks of 6 subjects and storing them in a list
marks = []
for i in range(1, 7):
   while True:
       try:
           mark = float(input(f"Enter marks for subject {i}: "))
           if 0 <= mark <= 100:   # check if mark is valid
               marks.append(mark)
               break
           else:
               print("Invalid input! Marks must be between 0 and 100. Try
again.")
       except ValueError:
           print("Invalid input! Please enter a number.")

# Calculating total, average, and percentage
total_marks = sum(marks)
average_marks = total_marks / len(marks)
percentage = (total_marks / (len(marks) * 100)) * 100

# Determining grade based on percentage

if percentage >= 85:
   grade = "Distinction"
elif percentage >= 70:
   grade = "First Division"
elif percentage >= 55:
   grade = "Second Division"
elif percentage >= 45:
   grade = "Third Division"
else:
   grade = "Fail"

# Displaying results
print(f"\nTotal Marks: {total_marks:.2f}")
print(f"Average Marks: {average_marks:.2f}")
print(f"Percentage: {percentage:.2f}%")
print(f"Grade: {grade}")
```

Output:

```
Enter marks for subject 1: 90
Enter marks for subject 2: 80
Enter marks for subject 3: 70
Enter marks for subject 4: 60
Enter marks for subject 5: 50
Enter marks for subject 6: 40

Total Marks: 390.00
Average Marks: 65.00
Percentage: 65.00%
Grade: Second Division
```

6. Write a program to display all **prime numbers within a user-defined range**. Additionally, print the **count of prime numbers** in that range along with their sum. **Marks [**10]

```
# Function to check if a number is prime
def is_prime(num):
    if num <= 1:
        return False
    for i in range(2, int(num ** 0.5) + 1): # checking for factors from 2
to the square root of the number
        if num % i == 0:
            return False
    return True

# Function to get prime numbers in a given range
def primes_in_range(start, end):
    prime_numbers = []
    for num in range(start, end + 1):
        if is_prime(num):
            prime_numbers.append(num)
    return prime_numbers

# Taking input for the range
while True:
    try:
        start_range = int(input("Enter the starting number of the range:
"))
        end_range = int(input("Enter the ending number of the range: "))
```

```
        if start_range <= end_range:
            break
        else:
            print("Invalid input! Starting number must be less than or
equal to ending number. Try again.")
    except ValueError:
        print("Invalid input! Please enter an integer.")


# Displaying results
primes = primes_in_range(start_range, end_range)
print(f"Prime numbers in the range {start_range} to {end_range}:
{primes}")
print(f"Count of prime numbers: {len(primes)}")
print(f"Sum of prime numbers: {sum(primes)}")
```

Output:

```
Enter the starting number of the range: 20
Enter the ending number of the range: 100
Prime numbers in the range 20 to 100: [23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
Count of prime numbers: 17
Sum of prime numbers: 983
```

7. Write a program to find all numbers between **1000 and 2500** (inclusive) that are divisible by 9 but not by 6. Extend the program so that the range limits (1000 and 2500) can also be entered by the user. **Marks [**10]

```
# Displaying numbers divisible by 9 but not by 6 between 1000 and 2500
print("Number divisible by 9 but not by 6 between 1000 and 2500
(inclusive):")
for num in range(1000, 2501):
    if num % 9 == 0 and num % 6 != 0:
        print(num)
print()

# Taking input for range limits
while True:
    try:
```

```
        # Taking input for the starting and ending numbers of the range and
validating them to ensure that the starting number is less than or equal
to the ending number
        start_range = int(input("Enter the starting number of the range:
"))
        end_range = int(input("Enter the ending number of the range: "))
        if start_range <= end_range: # if the starting number is less than
or equal to the ending number, break out of the loop and proceed to
display the numbers in the specified range
            break
        else: # if the starting number is greater than the ending number,
display an error message and prompt the user to enter the numbers again
            print("Invalid input! Starting number must be less than or
equal to ending number. Try again.")
    except ValueError: # handling the case where the user input for the
starting or ending number is not a valid integer and prompting them to try
again
        print("Invalid input! Please enter an integer.")

# Displaying numbers divisible by 9 but not by 6 in the user-defined range
print(f"Number divisible by 9 but not by 6 between {start_range} and
{end_range} (inclusive):")
for num in range(start_range, end_range + 1):
    if num % 9 == 0 and num % 6 != 0: # checking if the number is divisible
by 9 and not divisible by 6, and if so, printing the number
        print(num)
```

Output:

```
2385
2403
2421
2439
2457
2475
2493

Enter the starting number of the range: 9
Enter the ending number of the range: 90
Number divisible by 9 but not by 6 between 9 and 90 (inclusive):
9
27
45
63
81
```

8. Write a program to compute the factorial of a number entered by the user. Ensure the input is a valid positive integer. If the user enters anything else, display an error message such as **"Invalid input! Please enter a positive integer." Marks [**10]

```
def factorial(n): # function to calculate factorial of a number
   if n == 0 or n == 1: return 1
   else: return factorial(n - 1) * n # using recursion to calculate
factorial of n by multiplying n with the factorial of (n-1) until it
reaches the base case where n is 0 or 1, which returns 1.

# Taking input for the number and validating it
while True:
   try:
       num = int(input("Enter a positive integer to compute its factorial:
"))
       if num >= 0:
           break
       else:
           print("Invalid input! Please enter a positive integer. Try
again.")
   except ValueError:
       print("Invalid input! Please enter a positive integer. Try again.")

# Calculating and displaying the factorial
```

```
result = factorial(num)
print(f"The factorial of {num} is: {result}")
print()
```

Output:

```
Enter a positive integer to compute its factorial: 5
The factorial of 5 is: 120
```

9. Write a menu-driven program that repeatedly asks the user for numbers and computes the **separate sums of positive and negative numbers**. Continue until the user chooses to exit the program. **Marks [**10]

```
positive_sum = 0
negative_sum = 0

while True:
   # Displaying the menu options to the user and taking their choice as
input
   print("\nMenu:")
   print("1. Enter a number")
   print("2. Exit")
   choice = input("Choose an option (1 or 2): ")
   # Using a try-except block to handle invalid input and ensure the
program continues to run smoothly
   try:
       if choice == '1':
           num = float(input("Enter a number: ")) # taking input from the
user and converting it to a float to allow for decimal numbers
           if num > 0: # if the number is positive, add it to the
positive_sum
               positive_sum += num
           elif num < 0: # if the number is negative, add it to the
negative_sum
               negative_sum += num
       elif choice == '2': # if the user chooses to exit, display the sums
of positive and negative numbers and break out of the loop
           print(f"\nSum of positive numbers: {positive_sum}")
```

```
            print(f"Sum of negative numbers: {negative_sum}")
            print("Exiting the program. Goodbye!")
            break
        else: # if the user enters an invalid choice, prompt them to choose
again
            print("Invalid choice! Please choose 1 or 2.")
    except ValueError: # handling the case where the user input for the
number is not a valid float and prompting them to try again
        print("Invalid input! Please enter a valid number.")
```

Output:

```
Menu:
1. Enter a number
2. Exit
Choose an option (1 or 2): 1
Enter a number: 9090

Menu:
1. Enter a number
2. Exit
Choose an option (1 or 2): 1
Enter a number: -1949294

Menu:
1. Enter a number
2. Exit
Choose an option (1 or 2): 2

Sum of positive numbers: 9090.0
Sum of negative numbers: -1949294.0
Exiting the program. Goodbye!
```

10. Write a number guessing game in Python with the following rules: **Marks [** 2 * 5 = 10]

- The computer generates a random number between 1 and 50.
- The user must guess the number.
- After each guess, the program tells the user whether the guess is **too high, too low, or correct**.
- The user gets a maximum of **7 attempts**.
- If the user does not guess correctly within the attempts, display **"Better luck next time!"** and end the game.

```python
import random

max_attempts = 7

random_number = random.randint(1, 50) # generating a random number between 1 and 50 using the random module

current_attempt = 0 # variable to keep track of the current attempt number

print("Welcome to the Number Guessing Game!\nYou have 7 attempts to guess the number between 1 and 50. Good luck!\n")

print(f"(For testing purposes, the random number is: {random_number})\n")

while current_attempt < max_attempts:

   try:

       user_guess = int(input(f"Attempt no {current_attempt + 1}: Guess the number between 1 and 50: ")) # taking input from the user and converting it to an integer

       if(user_guess < 1 or user_guess > 50): # validating the user input to ensure it's between 1 and 50

           print("Invalid input! Please enter a number between 1 and 50. Try again.")

           continue

       # checking if the user's guess is too low, too high, or correct and printing the appropriate message

       if user_guess < random_number:

           print("Too low! Try again.")

       elif user_guess > random_number:

           print("Too high! Try again.")
```

```
        # if the guess is correct, congratulate the user and break out of
the loop

        else:

            print(f"Congratulations! You've guessed the number
{random_number} correctly in {current_attempt + 1} attempts!")

            break

        current_attempt += 1 # incrementing the attempt counter after each
valid guess

    except ValueError: # handling the case where the user input is not a
valid integer and prompting them to try again

        print("Invalid input! Please enter a valid integer. Try again.")



# if the user has used all attempts without guessing correctly, display
the "Better luck next time!" message along with the correct number

if current_attempt == max_attempts:

    print(f"Better luck next time! The correct number was
{random_number}.")
```

Output:

```
Welcome to the Number Guessing Game!
You have 7 attempts to guess the number between 1 and 50. Good luck!

(For testing purposes, the random number is: 21)

Attempt no 1: Guess the number between 1 and 50: 1
Too low! Try again.
Attempt no 2: Guess the number between 1 and 50: 60
Invalid input! Please enter a number between 1 and 50. Try again.
Attempt no 2: Guess the number between 1 and 50: 21
Congratulations! You've guessed the number 21 correctly in 2 attempts!
```